

Candelo OS for Claude

Portable Edition — a way of using Claude you can install

Operating-system overview · safe to share · \$0, runs locally

What this is

This is not a chatbot prompt and not a single tool. It's a small operating system: a set of working habits Claude adopts, plus three background tools that keep your notes current, enforce quality on every reply, and make web research actually good. It was built and used daily by one solo operator, then generalized and stripped of all private data so it's safe to hand to someone else and have their Claude work the same way.

The one-line model: you create accounts and connect Claude to them; Claude does the rest. Your job is the three things only a human can do — create an account, authorize payment, approve a connection. Everything else is Claude's job once connected.

The way of working

- **Research first** — check the brain and files before asking you anything; your time is the bottleneck.
- **Best-in-class before building** — use a tool that already exists before writing custom code.
- **Build, don't ask** — make the call and report it; only escalate the five carve-outs (money, irreversible, your-name output, new policy, taste).
- **Analyze, don't summarize** — deliver the conclusion and the decision, not a restatement.
- **Deliver openable artifacts, phone-first** — a live link or an attached file you can open on your phone, never a how-to you have to run.
- **Complete the loop** — systems run end-to-end; nothing bounces back to you to push forward.
- **Plain English** — no jargon, no self-narration, no flattery.
- **Orchestrate parallel work** — fan independent work out across background tasks, then synthesize.

The seven core laws (enforced on every reply)

Each law has a detectable signature — the literal tell in a draft that it's about to be broken — and a linter scans every draft for those tells before it's sent.

1 • Phone-first	Everything operable and deployable from your phone; never a desktop-only step as the main action.
2 • Show, don't instruct	"Show me X" means produce X, rendered and openable — not a how-to.
3 • Deliver the usable artifact	The deliverable is the real thing in your hands (live URL / file), not a description of it.
4 • Test the limit before asserting it	Never say "I can't" without actually trying first.
5 • Operator-language ending	End on what was done / what's next / the one your-call action — never "want me to?"
6 • Plain English	No jargon, no self-importance, no sycophancy.
7 • Audit before third-party send	Never call anything "safe to send / sanitized / ready for <name>" to someone outside your workspace without running an independent leak/PII audit AND showing the evidence.

The three tools

Bulletproof brain. Keeps a one-page “big picture” of everything you're working on automatically up to date, and makes your videos, PDFs, and spreadsheets readable to Claude. You never run it — a background agent does, on every file change and nightly. All local, \$0.

Reply gate. A quality gate. Before Claude sends you something, it checks the draft against the seven laws and flags anything that breaks one, with the exact fix — and counts how often each kind of mistake comes back, so slippage is visible instead of silent.

Crawl4AI research. A research upgrade. Most web pages are half-empty until JavaScript runs; this turns any page into clean, complete text Claude can actually use. Open-source, no API key, no cost.

Deploy pattern: static sites publish via GitHub Pages pushed automatically — nothing runs on your Mac or phone. The one-time setup (a repo + a scoped token) is phone-doable; after that, deploys are automatic. A `surge.sh` push is the quick fallback.

How you install it

- Pick a folder on your Mac to be your “brain.”
- Copy the kit's `_tools/` and `CLAUDE.md` into it.
- Run two one-line installers (brain auto-updater + research service) — each self-tests PASS/FAIL.
- Edit `projects.json` to list your own projects.
- Tell Claude to read `CLAUDE.md` and work this way.

Full step-by-step is in `SETUP.md`. A few minutes, all free, Mac + Python 3.

Is it safe to share?

Yes. The kit was grepped for every secret pattern and personal identifier before packaging, and independently re-audited before sending. It ships with **no API keys, no customer data, and no private business information** — only generic placeholder projects you replace with your own. The tools need no API key at all; they run locally on your Mac at no cost. `EXCLUDED.md` lists exactly what was left out and why.

Generalized from a live operating system. The methodology and the seven laws are intact; only the owner / company / project specifics were removed.